

Self-assessment criteria

Checklist for your group to **honestly** evaluate where it stands on the big project (distributed MapReduce system). Tick only what is **actually** done and working. A half-done criterion is not ticked. Three levels: **[Base]** = minimum expected · **[Solid]** = robust system · **[Advanced]** = goes further. Goal: spot your weak points and know where to put the remaining effort.

1. Infrastructure and deployment

- ☐ **[Base]** List of 100 **live** machines generated automatically (ajax.php API, canonical names ending in .enst.fr)
- ☐ **[Base]** Deployment script = **1 SCP** to the HOME (NFS) + **100 SSH** to start the servers
- ☐ **[Base]** Servers listen on a chosen **port**, with no collision with an already-used port
- ☐ **[Base]** **Cleaning** script that kills all deployed servers; clean redeployment possible
- ☐ **[Solid]** Deployment **robust** to unreachable machines (timeout, no blocking)
- ☐ **[Solid]** SSH with **keys** + **fingerprint bypass** configured (no password or "yes" to type)
- ☐ **[Solid]** Deployment and cleaning scripts **idempotent** and replayable

2. NFS vs local disk

- ☐ **[Base]** You know the HOME is on the **NFS** (not the local disk)
- ☐ **[Solid]** The **map intermediate files** are written to the **LOCAL disk** (e.g. /tmp), not the NFS
- ☐ **[Solid]** You deliberately limit the **load on the NFS** (controlled reads/writes)
- ☐ **[Advanced]** You **explored** the machines for local storage **beyond** /tmp (other partitions / scratch space, via df / mount) and use it appropriately

3. Protocol design

- ☐ **[Base]** **Main / Workers** roles clearly defined
- ☐ **[Base]** **Space-time diagram** of the V1 (without fault tolerance, KISS) done (sequencediagram.org)
- ☐ **[Solid]** Synchronization of the map → shuffle → reduce phases working
- ☐ **[Solid]** System **bootstrap** handled cleanly
- ☐ **[Advanced]** Protocol **formalized and documented** (messages, formats, edge cases)

4. MapReduce core

- ☐ **[Base]** **Word frequency** works on small splits (proof of concept)

- ☐ **[Base]** Key distribution at reduce via `hash(key) mod N`
- ☐ **[Solid]** **Remote read** of local intermediate files for the reduce phase
- ☐ **[Solid]** Results **validated** against a **single-machine reference** computation on the same small dataset (sanity checks: input vs output, identical counts)
- ☐ **[Advanced]** Works on **real Common Crawl splits** at scale (100, 1000+)

5. Common Crawl data

- ☐ **[Base]** Reading splits from `/cal/commoncrawl`
- ☐ **[Solid]** Understanding of the **structure** of Common Crawl
- ☐ **[Advanced]** **Direct read from Amazon** (without going through the internal NFS)

6. Performance and Amdahl's law

- ☐ **[Base]** **Timing** of every step (deployment, cleaning, I/O, sync/waiting, network, compute)
- ☐ **[Solid]** **Bottlenecks** identified and **backed by measurements**
- ☐ **[Solid]** **Reference point** established (speedup = 1 for 1 node)
- ☐ **[Solid]** **Speedup vs number of nodes** graph with the **same dataset** at every point
- ☐ **[Advanced]** You know when **projection/interpolation** is acceptable (single machine + strong linear hypothesis) and when it is **not** (two or more nodes → measure, never extrapolate)
- ☐ **[Advanced]** Amdahl's law **verified empirically** and **interpreted** (sequential part identified)

7. Fault tolerance

- ☐ **[Base]** **Detection** of a dead worker (ping / timeout)
- ☐ **[Solid]** **Re-execution** of lost tasks by the Main
- ☐ **[Solid]** **Atomic** output writes (temporary file + rename)
- ☐ **[Advanced]** **Demonstration** of tolerance by killing nodes mid-computation (as in the Google paper)
- ☐ **[Advanced]** Handling of **stragglers** (backup tasks)

8. Comparison and Kafka Streams (light)

Secondary: little time planned for Kafka. Do not spend most of the effort on it.

- ☐ **[Base]** You can situate **batch vs stream**
- ☐ **[Solid]** **Minimal wordcount** with Kafka Streams (downloaded files, no Docker or root)
- ☐ **[Solid]** **Documented** comparison vs Hadoop & Kafka Streams (no need to run Hadoop — e.g. note the missing **HDFS** and other features your simplified system lacks)
- ☐ **[Advanced · optional]** **Direct** Common Crawl read via a **Kafka source / connector** (instead of the overloaded NFS)

9. Use cases, report and demo

- ☐ **[Base] 3 use cases** beyond wordcount (languages, popularity, ranking, size, density...)
 - ☐ **[Solid]** Use case results **interpreted and explained**
 - ☐ **[Base] Report** covering the 7 required points: architecture · protocol · metrics · Amdahl's graph · pain points · 3 use cases · batch/stream comparison
 - ☐ **[Base] Demo** ready (10 min presentation + 10 min questions), work **distributed** between members
-

Group wrap-up

- How many **[Base]** criteria are ticked? (these are the non-negotiables)
- Is the work **fairly shared** and is the **atmosphere / collaboration** healthy? Does everyone feel involved?
- Is someone clearly **owning the Kafka Streams** part, so it is not left to the last minute?
- What are your **2 weakest points** today, and who is taking care of them?
- What **broke** and **why**? (keep this for the "pain points" section of the report)
- If the demo were tomorrow, what **would not pass**?